

DOCUMENTAÇÃO TÉCNICA DO PROJETO

Digital Corporativo Data Analytics

Pipeline de dados, análise de indicadores, dashboard interativo e camada de Inteligência Artificial

Tecnologias	Python • PostgreSQL • SQLAlchemy • Pandas • Streamlit • Plotly • IA
Ambiente	Visual Studio Code + ambiente virtual (venv)
Fonte	PostgreSQL — database datadt_digital_corporativo
Objetivo	Transformar dados corporativos em indicadores e insights para apoio à decisão

Documento elaborado como guia técnico e didático do desenvolvimento do projeto.

Sumário

1. Visão geral e objetivo
2. Arquitetura da solução
3. Estrutura de arquivos
4. Fonte de dados e modelo observado
5. Conexão com PostgreSQL
6. Extração dos dados
7. Transformação e tratamento
8. Análise e indicadores
9. Dashboard com Streamlit
10. Integração com Inteligência Artificial
11. Segurança e boas práticas
12. Fluxo de execução
13. Próximas etapas
14. Glossário

1. Visão geral e objetivo

O projeto Digital Corporativo — Data Analytics tem como finalidade construir uma solução de análise de dados corporativos a partir de um banco PostgreSQL. A proposta é criar um fluxo completo, desde a conexão e extração até o tratamento, cálculo de indicadores, visualização em dashboard e geração de recomendações com Inteligência Artificial.

O desenvolvimento é realizado em Python no Visual Studio Code, com ambiente virtual. A arquitetura foi organizada em módulos para separar responsabilidades, facilitar testes e permitir evolução.

Objetivo central: transformar dados operacionais de vendas, financeiro, clientes e recursos humanos em informação analítica para apoio à tomada de decisão.

A definição final de todos os KPIs e regras de negócio ainda faz parte da etapa de desenvolvimento.

Áreas identificadas

- **Vendas e faturamento.**
- **Financeiro:** contas a pagar e receber.
- **Clientes e pessoas.**
- **RH e vendedores.**
- **Produtos, categorias e formas de pagamento.**

2. Arquitetura da solução

A solução foi estruturada como um pipeline de Data Analytics, evitando concentrar conexão, SQL, tratamento, regras de negócio e interface em um único arquivo.

```
PostgreSQL
↓
database.py
↓
extraction.py
↓
transformation.py
↓
analysis.py
■■■■■■■■■■→ app.py → Dashboard
■■■■■■■■■■→ ai_analysis.py → IA → Recomendações
```

Cada camada possui uma responsabilidade:

- **Fonte:** PostgreSQL com schemas financeiro, geral, rh e vendas.
- **Conexão:** SQLAlchemy + psycopg2.
- **Extração:** consultas SQL.
- **Transformação:** Pandas.
- **Análise:** KPIs, rankings e agregações.
- **Apresentação:** Streamlit.
- **IA:** interpretação dos indicadores e recomendações.

3. Estrutura de arquivos

Arquivo	Responsabilidade
app.py	Interface do dashboard Streamlit e orquestração da aplicação.
.env	Credenciais e configurações sensíveis.
requirements.txt	Bibliotecas necessárias para executar o projeto.
.gitignore	Arquivos que não devem ser versionados.
src/__init__.py	Organiza src como pacote Python.
src/database.py	Cria conexão/engine com PostgreSQL.
src/extraction.py	Executa consultas e extrai dados.
src/transformation.py	Realiza tratamento e preparação com Pandas.
src/analysis.py	Centraliza KPIs, agregações e regras analíticas.
src/ai_analysis.py	Integração com API de IA e geração de recomendações.

4. Fonte de dados e modelo observado

Na exploração do banco foi identificado o database **datadt_digital_corporativo**. A conexão foi validada e a aplicação conseguiu consultar os schemas.

Schemas encontrados: financeiro, geral, rh e vendas.

Principais tabelas observadas:

- **financeiro:** conta_pagar, conta_receber, situacao_titulo e contas_receber_vendas.
- **geral:** pessoa, pessoa_fisica, pessoa_juridica, endereco, bairro, cidade, estado, contato e tipo_contato.
- **rh:** funcionario, cargo, departamento, escolaridade e lotacao.
- **vendas:** nota_fiscal, item_nota_fiscal, produto, categoria, parcela e forma_pagamento.

Também foram identificadas views para apoio a relatórios, como vendas.vi_lista_cliente, vendas.vi_relatorio_gestao, vendas.vi_vendas_vendedor, geral.vi_lista_pessoas e outras.

5. Conexão com PostgreSQL

O arquivo **src/database.py** cria a conexão do Python com PostgreSQL por meio do SQLAlchemy. As credenciais são lidas do **.env**, evitando gravá-las no código.

```
load_dotenv()
↓
os.getenv("DB_HOST"), os.getenv("DB_PORT")
os.getenv("DB_NAME"), os.getenv("DB_USER")
os.getenv("DB_PASSWORD")
↓
DATABASE_URL
↓
create_engine(...)
↓
engine
```

O objeto **engine** é reutilizado pelos demais módulos. A conexão foi testada com uma consulta simples retornando **(1,)** e, em seguida, foram listados schemas e tabelas.

6. Extração dos dados

O arquivo **src/extraction.py** concentra a extração. Foram realizadas consultas exploratórias para conhecer schemas, tabelas, colunas, tipos e relacionamentos.

Um teste de vendas + clientes retornou número da nota fiscal, data, valor, ID do cliente e nome do cliente, confirmando a navegação pelos relacionamentos.

```
SELECT
nf.id,
nf.numero_nf,
nf.data_venda,
nf.valor,
nf.id_cliente
FROM vendas.nota_fiscal nf
... JOINS necessários ...
```

Também foi investigado o relacionamento de vendedores. Foram encontrados IDs presentes em vendas.nota_fiscal, incluindo o ID 728. A investigação mostrou a necessidade de relacionar **vendas.nota_fiscal.id_vendedor** com **rh.funcionario.id** e, a partir de **id_pessoa**, chegar aos dados da pessoa.

7. Transformação e tratamento

O arquivo **src/transformation.py** transforma os dados extraídos em datasets analíticos. A camada prepara os dados para que os KPIs sejam calculados sobre informações consistentes.

Tratamentos previstos:

- Conversão e padronização de datas.
- Conversão de valores monetários para tipos numéricos.
- Tratamento de nulos conforme regra de negócio.
- Padronização de textos e categorias.
- Investigação de duplicidades.
- Validação de chaves após JOINS.
- Criação de colunas derivadas quando necessárias.
- Preparação dos DataFrames finais.

Valores ausentes ou inconsistentes devem ser investigados antes de serem simplesmente substituídos.

8. Análise e indicadores

O arquivo **src/analysis.py** concentra as regras analíticas. Ele recebe dados preparados e produz totais, rankings, agregações e indicadores para o dashboard.

Indicadores previstos:

- **Vendas:** faturamento, evolução temporal, categorias, vendedores, clientes e ticket médio.
- **Produtos:** receita, custo, margem e produtos mais rentáveis.
- **Clientes:** quantidade, PF/PJ e ranking.
- **Financeiro:** contas a pagar/receber, vencidos, situação e formas de pagamento.
- **RH/Vendedores:** relacionamento de funcionário, pessoa, cargo, departamento e desempenho de vendas.

9. Dashboard com Streamlit

O arquivo **app.py** é a camada de apresentação. O Streamlit transforma os resultados analíticos em uma aplicação web executada localmente.

O dashboard deve consumir funções dos módulos anteriores. Consultas SQL e tratamentos complexos não devem ficar concentrados no app.py.

Estrutura sugerida:

- **Visão executiva:** KPIs principais.
- **Vendas:** evolução, categorias, vendedores, clientes e produtos.
- **Financeiro:** contas e títulos.
- **Clientes:** perfil e concentração.
- **Insights:** recomendações da IA.

Plotly pode ser usado para gráficos interativos.

10. Integração com Inteligência Artificial

O arquivo **src/ai_analysis.py** separa a integração com IA da lógica do dashboard. A função recebe indicadores já calculados e solicita uma interpretação executiva.

```
KPIs
↓
ai_analysis.py
↓
API de IA
↓
interpretação
↓
recomendações
↓
Streamlit
```

Durante o desenvolvimento foi usada inicialmente uma chave com prefixo **xai-** enquanto o código utilizava o SDK da Groq, causando erro 401. A credencial deve ser compatível com o provedor configurado no código.

A IA deve interpretar indicadores confiáveis; não deve inventar números, substituir cálculos ou receber credenciais do banco.

11. Segurança e boas práticas

Credenciais do PostgreSQL e chaves de API devem permanecer no **.env**, que precisa estar no **.gitignore**.

- Nunca publicar senhas ou chaves no GitHub.
- Nunca compartilhar uma chave completa em mensagens ou documentação pública.
- Usar variáveis de ambiente.
- Rotacionar/revogar credenciais expostas.
- Separar código, configuração e dados.

Recomendação: credenciais reais compartilhadas durante o desenvolvimento devem ser alteradas/rotacionadas, principalmente se forem de produção.

12. Fluxo de execução

1. Ativar ambiente virtual
2. Instalar requirements.txt
3. Configurar .env
4. Testar database.py

```
5. Testar extraction.py
6. Testar transformation.py
7. Testar analysis.py
8. Testar ai_analysis.py
9. Executar Streamlit
10. Validar KPIs e dashboard

python -m pip install -r requirements.txt

python -c "from src.database import engine; print('DATABASE OK!)"

python -m streamlit run app.py
```

A separação modular facilita localizar falhas: conexão em database.py; consultas em extraction.py; qualidade em transformation.py; indicadores em analysis.py; interface em app.py.

13. Próximas etapas

1. Finalizar os JOINS de vendas, clientes, produtos, categorias, vendedores e financeiro.
2. Criar os datasets analíticos definitivos.
3. Executar tratamento e validação de qualidade dos dados.
4. Definir formalmente os KPIs e suas fórmulas.
5. Criar funções analíticas reutilizáveis.
6. Construir o dashboard Streamlit com layout executivo.
7. Adicionar filtros e gráficos Plotly.
8. Conectar a IA aos KPIs finais.
9. Testar consistência entre banco, DataFrames e dashboard.
10. Documentar o projeto e preparar README para GitHub/portfólio.

14. Glossário

- **ETL:** Extract, Transform, Load — extração, transformação e carregamento.
- **DataFrame:** estrutura tabular do Pandas.
- **JOIN:** operação SQL para relacionar tabelas.
- **KPI:** indicador-chave de desempenho.
- **Pipeline:** sequência de etapas do dado à informação.
- **Schema:** agrupamento lógico de objetos do banco.
- **SQLAlchemy:** biblioteca Python para interação com bancos.
- **Streamlit:** framework Python para aplicações de dados.
- **Plotly:** biblioteca para visualizações interativas.
- **API Key:** credencial de autenticação de uma API.

Conclusão

O projeto foi estruturado para demonstrar uma cadeia completa de Data Analytics: acesso a uma fonte relacional, exploração do modelo, extração, tratamento em Python, indicadores, visualização e uso responsável de IA para interpretação. A separação em camadas cria uma base adequada para testes, manutenção e evolução do portfólio.